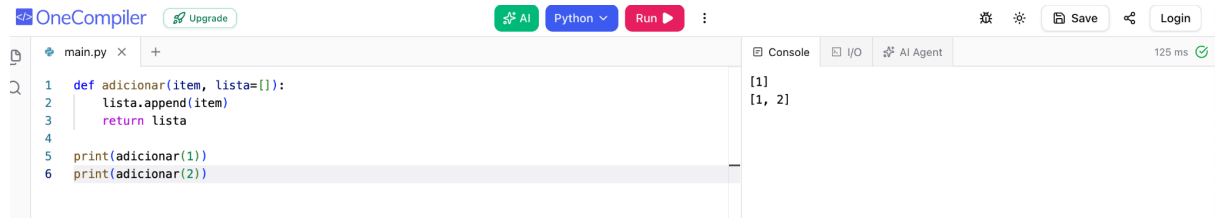


Paradigmas e Linguagens de Programação

Atividade aula-008 - Subprogramas

Heitor Henrique Scramim de Freitas - RA: 24190382-2

ex1: Python



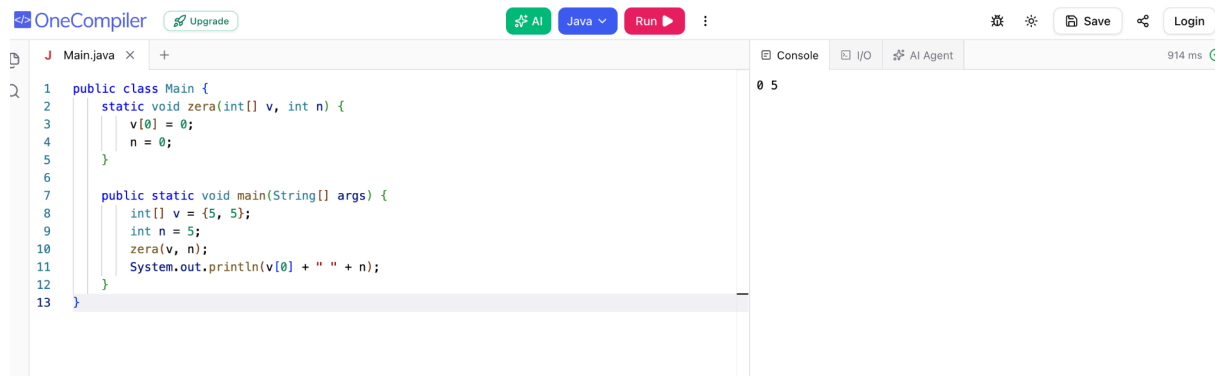
```
def adicionar(item, lista=[]):
    lista.append(item)
    return lista

print(adicionar(1))
print(adicionar(2))
```

Console: [1], [1, 2]

- a)
- b) Parâmetro com valor padrão combinado com passagem por atribuição do Python. O valor padrão [] é criado uma única vez, na definição da função, e não a cada chamada. Como a lista é mutável, as duas chamadas compartilham o mesmo objeto (um apelido). Correção: lista=None e if lista is None: lista = [].

ex2: Java



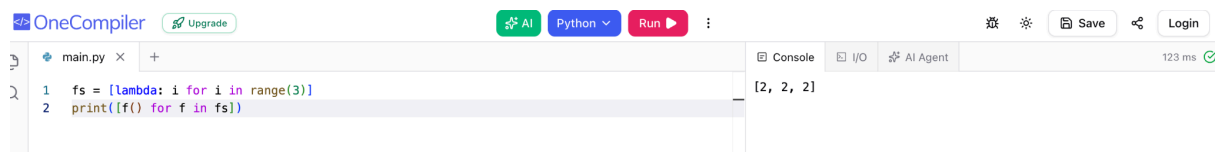
```
public class Main {
    static void zera(int[] v, int n) {
        v[0] = 0;
        n = 0;
    }

    public static void main(String[] args) {
        int[] v = {5, 5};
        int n = 5;
        zera(v, n);
        System.out.println(v[0] + " " + n);
    }
}
```

Console: 0 5

- a)
- b) Em Java tudo é passado por valor. O n (primitivo) é uma cópia, então o chamador nunca vê a alteração. O v é uma cópia da referência, que aponta para o mesmo array, então v[0] = 0 altera o objeto compartilhado.

ex3: Python



```
fs = [lambda: i for i in range(3)]
print([f() for f in fs])
```

Console: [2, 2, 2]

- a)
- b) Fechamentos capturam a variável (a célula de memória), e não uma cópia do valor. É o mesmo caso do var i em JavaScript, que dá [3, 3, 3]. Existe um único i, que vale 2 quando as lambdas são chamadas. Correção: lambda i=i: i, que dá [0, 1, 2].

ex4: C

```

1 #include <stdio.h>
2
3 int contador(void) {
4     static int n = 0;
5     return ++n;
6 }
7
8 int main(void) {
9     contador();
10    contador();
11    printf("%d\n", contador());
12    return 0;
13 }

```

Console output: 3

- a)
- b) variável local estática (static) nos ambientes de referenciamento local. Ela fica na área estática, e não na pilha, então lembra o valor entre as chamadas. Sem o static (dinâmica da pilha), a saída seria 1.

ex5: Rust

```

1 fn dobra(v: Vec<i32>) -> Vec<i32> {
2     v.iter().map(|x| x * 2).collect()
3 }
4
5 fn main() {
6     let v = vec![1, 2, 3];
7     let d = dobra(v);
8     println!("{:?} {:?}", v, d);
9 }

```

Console output: error[E0382]: borrow of moved value: `v`
 --> HelloWorld.rs:8:27
 6 | let v = vec![1, 2, 3];
 | - move occurs because `v` has type `Vec<i32>`, which does not implement the `Copy` trait
 7 | let d = dobra(v);
 | - value moved here
 8 | println!("{:?} {:?}", v, d);
 | ^ value borrowed here after move
 note: consider changing this parameter type in function `dobra` to borrow instead if owning the value isn't necessary
 --> HelloWorld.rs:1:13
 1 | fn dobra(v: Vec<i32>) -> Vec<i32> {
 | ~~~~~ ~~~~~~ this parameter takes ownership of the value
 | |
 | in this function
 help: consider cloning the value if the performance cost is acceptable
 7 | let d = dobra(v.clone());
 | +++++++
 error: aborting due to 1 previous error
 For more information about this error, try `rustc --explain E0382`.

- a)
- b) Em Rust a passagem padrão é por posse (move). Ao chamar dobra(v), a posse do vetor vai para a função e v deixa de ser válido no main. Usá-lo no println! é erro de compilação.
- c) Versão corrigida:

```

1 fn dobra(v: &Vec<i32>) -> Vec<i32> {
2     v.iter().map(|x| x * 2).collect()
3 }
4
5 fn main() {
6     let v = vec![1, 2, 3];
7     let d = dobra(&v);
8     println!("{:?} {:?}", v, d);
9 }

```

Console output: [1, 2, 3] [2, 4, 6]

ex6: Python

The screenshot shows the OneCompiler Python IDE interface. On the left, a code editor displays a Python script named `main.py` with the following content:

```
1 total = 0
2
3 def adiciona(x):
4     total = total + x
5     return total
6
7 print(adiciona(5))
```

On the right, the console shows a traceback for an `UnboundLocalError`:

```
Traceback (most recent call last):
  File "/box/main.py", line 7, in <module>
    print(adiciona(5))
    ~~~~~
  File "/box/main.py", line 4, in adiciona
    total = total + x
    ~~~~~
UnboundLocalError: cannot access local variable 'total' where it
is not associated with a value
```

The error message indicates that the function `adiciona` is trying to modify a local variable `total` that has not been assigned a value yet.

- a)
- b) Ambiente de referenciamento local. Como há uma atribuição a `total` dentro da função, o Python trata `total` como variável local em toda a função. Ao ler `total + x`, ela ainda não tem valor. Para alterar a global, declare global `total` no início da função. Em funções aninhadas, o equivalente é `nonlocal` (visto na síntese, na linha do Python). A alternativa sem efeito colateral é receber o `total` como parâmetro e devolver o novo valor.